

FitnessAiClient - Architektur & Implementierungsplan

Prompt: FitnessAiClient – Architektur & Implementierungsplan

Kontext

Grundprinzipien

API

Authentifizierung (JWT / Refresh Token)

HealthConnectService

Snapshot-Modell (Kernkonzept für Datenversand)

Payload-Struktur

Endpunkte

Storage

Scheduler

HTTP

Localization

Screens

Empfohlene Aufbau-Reihenfolge

Explizit außerhalb dieses Scopes

Bewusst vertagte Entscheidungen

Prompt: FitnessAiClient - Architektur & Implementierungsplan

Kontext

Neuimplementierung der React Native App **FitnessAiClient**, die Health-Connect-Daten via `react-native-health-connect` ausliest und an die **FitnessAI API** sendet. Kompletter Neubau (kein inkrementeller Umbau der bestehenden App). Der bestehende Auth-Flow dient als fachliche Referenz. Lokale Datenbank: **FitnessAiClientDb**.

Grundprinzipien

1. **Origin-Erhalt ist Pflicht:** Nutzer verwenden unterschiedliche Quellen (z. B. Zepp/Amazfit, Google Fit, Fitbit, Wear OS). `dataOrigin` muss pro Datenpunkt erhalten bleiben und an die API übertragen werden.
 2. **Client sendet Rohdaten, Backend entscheidet/aggregiert:** Keine Mapping-, Filter- oder Aggregationslogik im Client.
 3. **Snapshot- statt Delta-Modell** (siehe unten) – löst das Problem nachträglich korrigierter/verspätet synchronisierter Wearable-Daten strukturell, ohne Health-Connect-Changes-API oder Cursor-Tracking.
 4. **Performance:** Kein Tages-Loop beim Health-Connect-Abwurf. Ein Request pro Zeitraum, Pagination via `pageToken`.
-

API

- **FitnessAI API** als einzige Datensinke
 - Zentraler Axios-Client, alle Endpunkte als typisierte Service-Funktionen (keine verstreuten `axios.get/post`-Aufrufe in Screens/Workern)
-

Authentifizierung (JWT / Refresh Token)

- Bestehende Auth-Logik (Login-Request, Token-Handling, Refresh-Mechanismus) ist **fachlich bereits gegen die API verifiziert** und dient als Referenz – Verhalten wird übernommen, Code darf strukturell an die neue Architektur angepasst werden.
- **JWT + RefreshToken** → **UserAuthenticationTable** (lokale DB, siehe Kapitel „Storage“), **feld-verschlüsselt** (AES-256-GCM): Der Verschlüsselungs-Key liegt im Keychain/Keystore, einmalig generiert – das ist der einzige verbleibende Secure-Storage-Zugriff der App. `jwt / refreshToken` werden vor dem Schreiben verschlüsselt (Ciphertext + IV), beim Lesen wieder entschlüsselt. Library z. B. `react-native-aes-crypto` oder `expo-crypto` für die AES-Operationen, `react-native-keychain` für den Key selbst.
- Axios-Interceptor:
 - Request: liest `jwt` aus `UserAuthenticationTable`, entschlüsselt es, hängt es an
 - Response: bei `401` → Refresh-Flow, Original-Request danach wiederholen, neue Werte für `jwt / refreshToken / expiresAt` verschlüsselt in `UserAuthenticationTable` schreiben
 - **Mutex/Queue zwingend:** Bei parallelen Requests (UI + Background- Scheduler) darf nur ein Refresh-Call gleichzeitig laufen; alle anderen warten auf dessen Ergebnis
 - Schlägt Refresh fehl → `UserAuthenticationTable`-Eintrag löschen, zurück zu `LoginPage`
 - **Key-Verlust-Handling:** Schlägt die Entschlüsselung von `jwt` fehl (z. B. weil der Keychain-Key durch Nutzer-Reset/Geräte-Wiederherstellung nicht mehr existiert), wird das wie ein abgelaufenes Refresh-Token behandelt: `UserAuthenticationTable`-Eintrag löschen, zurück zu `LoginPage`, erneuter Login erzeugt automatisch einen neuen Key. Andere App-Daten (Schedules, Health-Snapshot-Cache) sind davon nicht betroffen, da nur diese eine Tabelle feld-verschlüsselt ist.

Login Flow (Post-Auth)

```

User meldet sich an (API-Request)
├─ Erfolg:
  1. UserDataTable von der API laden, lokal speichern (siehe Storage → UserDataTable)
  2. HealthConnectService.checkAvailability() + .requestPermissions()
     aufrufen (Verfügbarkeits-Check + Datentyp-Permission-Request; Details
     siehe Kapitel „HealthConnectService“)
     → nicht verfügbar: Weiterleitung Play-Store-Eintrag, Flow pausiert
     → verfügbar: weiter zur Tab-Navigation
└─ Fehlschlag:
   Fehlermeldung anzeigen, User bleibt auf LoginPage

```

Die `READ_HEALTH_DATA_IN_BACKGROUND`-Permission wird vom `HealthConnectService` separat behandelt (siehe Kapitel „Scheduler“) – sie wird erst beim ersten Aktivieren eines Schedules angefordert, nicht bereits im Login-Flow.

HealthConnectService

Eigenständiges Modul, kapselt sämtliche Interaktion mit `react-native-health-connect`. Keine andere Stelle der App (Login-Flow, Scheduler, Dashboard) spricht direkt mit der Library – alle greifen über diesen Service.

Verantwortlichkeiten:

- **Verfügbarkeits-Check:** Ist Health Connect auf dem Gerät installiert? Falls nicht → Signal an aufrufende Stelle (z. B. Login-Flow leitet zum Play-Store-Eintrag weiter)
- **Permission-Request** (Datentyp-Permissions: Steps, HeartRate, ExerciseSession, ...) – wird direkt nach erfolgreichem Login angestoßen (siehe Login Flow)
- **Background-Permission-Request** (`READ_HEALTH_DATA_IN_BACKGROUND`) – separat, ausgelöst beim ersten Aktivieren eines Schedules (siehe Kapitel „Scheduler“)
- **Read-Layer:** `readRecords` + Pagination (siehe unten)
- **Permission-Revocation-Handling:** Jeder `readRecords`-Aufruf (Local Refresh, Initial Load) ist gegen fehlende/entzogene Permissions abgesichert (try/catch). Schlägt ein Read deswegen fehl, wird das im Scheduler-Tab als Fehlerstatus angezeigt („Berechtigung fehlt – bitte erneut erteilen“) mit einem Button, der erneut `HealthConnectService.requestPermissions()` aufruft. Kein automatisches Re-Prompt ohne Nutzerinteraktion (Android verlangt ohnehin eine explizite Nutzeraktion für Permission-Dialoge).

Read-Layer

- Nutzt `readRecords` (**nicht** `aggregateGroupByPeriod`) – Aggregation würde `dataOrigin` zerstören
- Ein Request pro angefragtem Zeitraum + vollständige Pagination via `pageToken`
- Jeder gelesene Record wird inkl. `metadata.id` (Health-Connect-eigene Record-ID), `dataOrigin`, Timestamp/Zeitfenster und Wert(en) weiterverarbeitet – die `metadata.id` wird für Backend-seitige Idempotenz benötigt (siehe Snapshot-Modell)

Snapshot-Modell (Kernkonzept für Datenversand)

Statt Delta-/Cursor-basiertem Sync wird bei jedem Lauf ein **kompletter Zeitraum neu gelesen und gesendet**; das Backend **ersetzt** (upsert) den Datenbestand für diesen Zeitraum, statt zu addieren. Das erfasst nachträglich korrigierte oder verspätet synchronisierte Wearable-Daten zuverlässig, ohne Cursor/Changes-API.

Lauf-Typ	Zeitraum	Auslösung
Initial Load	(heute - 365 Tage) bis heute	einmalig, manuell getriggert (nicht Teil von <code>ScheduleTable</code>)
Scheduled Sync	(heute - 1 Tag) bis heute	wiederkehrend, gesteuert über <code>ScheduleTable</code>

Bekannter Trade-off (bewusst akzeptiert): Korrekturen/Nachlieferungen, die älter als das jeweilige Fenster sind, werden nicht mehr erfasst. Für Scheduled Sync (1 Tag) ist das eine bewusste Entscheidung – falls sich das als zu eng erweist, ist das Fenster über einen zusätzlichen Parameter pro Schedule erweiterbar.

Backend-Voraussetzung (Scope: Backend-Ticket, aber beim Payload-Design zu berücksichtigen):

Idempotente Upsert-Keys: - Health-Daten (Steps, Calories, ...): `date` + `dataOrigin` + `recordType` - `ExerciseSession`:

Health-Connect-eigene `recordId` (`metadata.id`)

Payload-Struktur

Rohdaten, keine Aggregation, keine Origin-Filterung im Client:

- Record-Type
- Wert(e)
- Timestamp bzw. Start-/Endzeit (bei Sessions)
- `dataOrigin`
- `metadata.id` (Health-Connect-Record-ID, für Backend-Upsert)

Endpunkte

- **PostHealthData** – rohe, tägliche Health-Connect-Daten (Steps, Calories, HeartRate, ...) inkl. `dataOrigin` und `metadata.id`. Backend aggregiert und upserted selbst.
- **PostTrainingData** – rohe Exercise-Session-Daten inkl. zugehöriger Metrik-Records und `dataOrigin`. Backend berechnet Auswertung/Evaluation. → **Feld-Modell bewusst vertagt**, wird zu einem späteren Zeitpunkt final definiert (Optionen reichen von reinen Summen bis zu unaggregierten Metrik-Records im Session-Zeitfenster). `ExerciseTable` im lokalen Schema ist davon unabhängig nutzbar, da sie nur die Session selbst hält – die Verknüpfung mit Metrik-Records erfolgt erst beim Payload-Bau (siehe Storage → „Verknüpfung Exercise ↔ Metriken“)

Storage

Lokale SQLite-DB `FitnessAiClientDb` (z. B. `expo-sqlite`, `@op-engineering/op-sqlite`, oder `react-native-sqlite-storage` – kein spezieller verschlüsselter Build nötig). Enthält **alle** App-Daten – `UserDataTable`, `UserAuthenticationTable`, `SettingsTable`, `Schedules`, Initial-Load-Status und den Health-Connect-Snapshot-Cache.

jwt und refreshToken sind feld-verschlüsselt (AES-256-GCM), nicht die gesamte DB – siehe Kapitel „Authentifizierung“ für Details. Der Verschlüsselungs-Key liegt im Keychain/Keystore, einmalig generiert; das ist der einzige verbleibende Secure-Storage-Zugriff der App.

Kein reines Key-Value-Storage (AsyncStorage/MMKV) für die DB-Inhalte, da relationale Queries (Schedule-Lookup, Datums-/Zeitraum-Filter, Fortschritts-Tracking) benötigt werden.

Alle Tabellen außer `UserDataTable` referenzieren `userId` (siehe Schema unten) – relevant, falls sich auf demselben Gerät z. B. Nutzer A und Nutzer B (nacheinander) anmelden: Alle Queries (Schedule-Lookup, Heartbeat-Prüfung, Dashboard-Daten) filtern zwingend nach der aktuell eingeloggten `userId`, damit keine Daten zwischen Nutzern vermischt werden.

Feste Namenskonvention (gilt für jede Tabelle, auch künftig neue): – `id` ist immer der lokale Primary Key – `userId` ist immer der Foreign Key auf `UserDataTable.id`, außer in `UserDataTable` selbst

Diese Konvention ist bereits im aktuellen Schema konsequent umgesetzt (siehe unten) und sollte bei jeder neuen Tabelle beibehalten werden.

Foreign Keys werden als echte Constraints angelegt, nicht nur als Kommentar/Konvention – `ON DELETE CASCADE` auf `userId`, damit beim Löschen eines `UserDataTable`-Eintrags (z. B. Nutzerwechsel/Neuinstallation) alle abhängigen Zeilen automatisch mitgelöscht werden. Wichtig bei SQLite: Foreign Keys sind standardmäßig **deaktiviert** und müssen pro Connection explizit aktiviert werden (`PRAGMA foreign_keys = ON;`), sonst werden die Constraints zwar angelegt, aber nicht durchgesetzt.

DB-Schema

```
UserDataTable          -- geladen von der API nach erfolgreichem Login
├── id
└── firstName
```

```

└─ lastName
└─ email
└─ createdAt
└─ updatedAt

UserAuthenticationTable
└─ id                number, PK
└─ userId            number, FK -> UserDataTable.id, ON DELETE CASCADE, UNIQUE
└─ jwt              string  -- AES-256-GCM-verschlüsselt gespeichert
└─ refreshToken      string  -- AES-256-GCM-verschlüsselt gespeichert
└─ expiresAt         Date    -- Annahme: Ablaufzeit des jwt (Access Token);
                             refreshToken-Lebensdauer wird serverseitig
                             verwaltet, nicht separat lokal getrackt
└─ createdAt
└─ updatedAt

SettingsTable
└─ id
└─ userId            FK -> UserDataTable.id, ON DELETE CASCADE, UNIQUE
└─ lang              TEXT, CHECK IN ('en','de'), Default 'en'
└─ createdAt
└─ updatedAt

ScheduleTable
└─ id                INTEGER, PK
└─ userId            INTEGER, FK -> UserDataTable.id, ON DELETE CASCADE
└─ type              TEXT, CHECK: 'HealthDataExport' | 'ExerciseDataExport'
└─ intervalType      INTEGER, CHECK IN (0,1,2): 0=Hourly, 1=Daily, 2=Weekly
└─ day               INTEGER, NULL, CHECK 1-7 (ISO-8601, 1=Monday).
                             Nur relevant bei intervalType=2 (Weekly).
                             Bei 0/1 ungenutzt (NULL).
└─ hour              INTEGER, NULL, CHECK 0-23. Zielstunde bei
                             intervalType 1 (Daily) oder 2 (Weekly).
└─ minute            INTEGER, NULL, CHECK 0-59. Bei intervalType=0
                             (Hourly) Minute innerhalb der Stunde, sonst Teil
                             der Zielzeit zusammen mit hour.
└─ isActive          INTEGER, CHECK IN (0,1), Default 1
└─ lastRunAt         TEXT, NULL -- dient zugleich als Duplikat-Schutz
└─ lastSuccessAt     TEXT, NULL
└─ createdAt         TEXT, Default now
└─ updatedAt         TEXT, Default now

DailyHealthDataTable -- Health-Connect-Snapshot-Cache, Nicht-Exercise-Metriken
└─ id                -- lokaler PK
└─ userId            FK -> UserDataTable.id, ON DELETE CASCADE
└─ recordId          TEXT  -- Health-Connect metadata.id
└─ recordType        TEXT  -- 'Steps' | 'Calories' | 'HeartRate' | ...
└─ date              TEXT  -- Kalendertag, dem der Wert zugeordnet ist
└─ value             REAL   -- gemessener Wert
└─ dataOrigin        TEXT, NULL
└─ startTime         TEXT  -- Original-Zeitfenster des Records
└─ endTime           TEXT
└─ createdAt
└─ updatedAt         -- lokaler Schreibzeitpunkt, erkennt Korrekturen beim Upsert
UNIQUE (userId, recordId) -- zusammengesetzter Upsert-Key, s. Hinweis unten

ExerciseTable -- Health-Connect-Snapshot-Cache, ExerciseSession-Daten
└─ id                -- lokaler PK
└─ userId            FK -> UserDataTable.id, ON DELETE CASCADE
└─ exerciseId        TEXT  -- Health-Connect ExerciseSession metadata.id
                             (String/UUID – nicht INTEGER, s. Hinweis unten)
└─ dataOrigin        TEXT, NULL
└─ exerciseType      TEXT
└─ startTime         TEXT
└─ endTime           TEXT
└─ durationSeconds   REAL
└─ createdAt
└─ updatedAt         UNIQUE (userId, exerciseId) -- zusammengesetzter Upsert-Key, s. Hinweis unten

ExerciseValuesTable -- normalisierte Metrik-Werte pro Exercise-Session
└─ id                -- lokaler PK
└─ exerciseId        FK -> ExerciseTable.id, ON DELETE CASCADE
└─ valueType         TEXT  -- z.B. 'Calories' | 'AvgHeartRate' | 'Steps' | ...
└─ value             REAL
└─ unit              TEXT
└─ createdAt
└─ updatedAt

```

Verknüpfung Exercise ↔ Metriken - normalisiert statt Zeit-Overlap: `ExerciseValuesTable` hält die zu einer Session gehörenden Metrik-Werte (z. B. Calories, AvgHeartRate, Steps während der Session) direkt per FK auf `ExerciseTable.id`, befüllt beim Local Refresh zusammen mit der Session selbst. Ersetzt die ursprünglich geplante `startTime / endTime` -Overlap- Query gegen `DailyHealthDataTable` - sauberer modelliert (kein impliziter Zeitfenster-Match nötig) und einfacher abzufragen (JOIN über `exerciseId` statt Zeitraum-Vergleich). `DailyHealthDataTable` bleibt ausschließlich für die Nicht-Exercise-Tagesmetriken zuständig, es gibt keine Überschneidung mehr zwischen beiden Tabellen.

Zusammengesetzter Upsert-Key (`userId, recordId`) bzw. (`userId, exerciseId`), nicht die Health-Connect-ID allein: Health-Connect-IDs sind auf Geräte-Ebene eindeutig, nicht pro FitnessAiClient-Login. Melden sich User A und User B nacheinander auf demselben Gerät an, lesen beide potenziell dieselben Health-Connect-Records (gleiche ID). Ohne `userId` im Unique-Key würde der Local-Refresh eines Users versehentlich die lokale Zeile des anderen Users überschreiben. Der zusammengesetzte Key stellt sicher, dass jeder User seine eigene lokale Kopie desselben physischen Records hält.

exerciseId als TEXT, nicht INTEGER: Health-Connect-`metadata.id`s sind Strings/UUIDs, kein Integer – analog zu `recordId` in `DailyHealthDataTable`. Bitte beim Anlegen der Migration gegenprüfen, falls der aktuelle Tabellenentwurf hier abweicht.

Bewusst kein `syncStatus / pending`-Feld auf Record-Ebene: Da das Snapshot-Modell beim Send immer den kompletten aktuellen Fensterstand überträgt (Backend upserted ohnehin), reicht `lastRunAt / lastSuccessAt` auf Schedule-Ebene. Schlägt ein Send fehl, wird beim nächsten fälligen Lauf der volle (ggf. inzwischen aktualisierte) Stand erneut gesendet.

Audit-Spalten als Standard: Jede Tabelle erhält `createdAt / updatedAt` (Default `datetime('now')`), zusätzlich zur `id / userId`-Konvention. Gilt ebenfalls für jede künftig neue Tabelle.

CHECK-Constraints statt reiner Doku-Konvention: Enum-artige Felder (`type, intervalType, lang, isActive`) und Wertebereiche (`day, hour, minute`) werden per CHECK-Constraint auf DB-Ebene erzwungen, nicht nur im Code validiert.

Nicht übernommen: Ein zwischenzeitlich diskutiertes Feld `credentialsId` auf `UserDataTable` wird nicht ins Schema aufgenommen – redundant zur bestehenden `userId`-Beziehung in `UserAuthenticationTable`, ohne eigenen FK ohnehin nicht durchsetzbar.

Indizes

```
idx_authentication_userId      ON UserAuthenticationTable(userId)
idx_schedule_userId            ON ScheduleTable(userId)
idx_health_userId_date         ON DailyHealthDataTable(userId, date)
idx_exercise_userId_startTime  ON ExerciseTable(userId, startTime)
idx_exercise_values_exerciseId ON ExerciseValuesTable(exerciseId)
```

Scheduler

Anforderung

Muss auch bei vollständig beendeter App im eingestellten Intervall laufen.

Zwingende technische Voraussetzung

Permission `android.permission.health.READ_HEALTH_DATA_IN_BACKGROUND` im Manifest deklarieren + zur Laufzeit vom Nutzer genehmigen lassen. Ohne diese Permission schlagen Health-Connect-Reads fehl, sobald die App im Hintergrund oder beendet ist.

Implementierungsoptionen (Prototyp-Entscheidung)

Beide Optionen werden als Prototyp umgesetzt und gegeneinander evaluiert, bevor final entschieden wird – kein Vorab-Commitment auf eine Option.

- **Option A - Natives WorkManager (Kotlin):** `CoroutineWorker` + `WorkManager.enqueueUniquePeriodicWork(...)`. OS-verwaltet, unabhängig vom JS-Prozess. API-Call müsste nativ erfolgen oder einen JS-Einstiegspunkt triggern.

- **Option B - Headless JS + natives Scheduling:** Native Komponente weckt JS-Kontext im Hintergrund (z. B. via `react-native-background-fetch` oder vergleichbar), bestehende JS-Logik (Health-Connect-Read + API-Call) läuft unverändert weiter.

Evaluationskriterien für die Prototypen: Zuverlässigkeit bei “App vollständig beendet” über mehrere Stunden/Tage (insbesondere auf Geräten mit aggressivem Battery-Management, z. B. Xiaomi/Huawei), tatsächlich erreichbare Heartbeat-Genauigkeit, Implementierungsaufwand für den API-Call-Pfad (nativ vs. JS), Wartbarkeit.

Hinweis: “Force Stop” durch den Nutzer stoppt bei beiden Optionen jeden Hintergrundbetrieb – OS-Grenze, kein Implementierungsfehler.

Ausführungsmodell: Heartbeat-Pattern

Periodische OS-Jobs können nicht exakt auf die Minute triggern (WorkManager- Minimum ca. 15 Min.). Daher: Ein Heartbeat-Job läuft alle 15–30 Minuten, führt bei jedem Tick den **Local Refresh** aus und prüft zusätzlich gegen `ScheduleTable`, welche Schedules gerade fällig sind (**Send**-Schritt).

Heartbeat-Logik (Auszug, pro Schedule-Zeile geprüft)

```
wenn intervalType == 0 (Hourly):
    fällig, wenn: aktuelle Minute >= minute
                  UND (jetzt - lastRunAt) >= ~1 Stunde

wenn intervalType == 1 (Daily):
    fällig, wenn: aktuelle Zeit >= {hour, minute}
                  UND Datum(lastRunAt) != heute

wenn intervalType == 2 (Weekly):
    fällig, wenn: aktueller Wochentag == day
                  UND aktuelle Zeit >= {hour, minute}
                  UND Datum(lastRunAt) != heute
```

Local Refresh vs. Send - getrennte Schritte pro Heartbeat-Tick

1. **Local Refresh** (bei *jedem* Heartbeat-Tick, unabhängig vom Schedule-Intervall): liest das Snapshot-Fenster (heute-1 bis heute) via `readRecords` aus Health Connect, **upserted** die Ergebnisse in `DailyHealthDataTable` (Match `userId + recordId`) bzw. `ExerciseTable` (Match `userId + exerciseId`) inkl. zugehöriger Werte in `ExerciseValuesTable`. Erfasst nachträgliche Korrekturen zuverlässig, ohne dass beim Send erneut Health Connect abgefragt werden muss.
2. **Send** (nur wenn laut `ScheduleTable` fällig, siehe Heartbeat-Logik): liest den aktuellen Stand direkt aus der lokalen Tabelle für das Fenster, sendet ihn komplett an `PostHealthData / PostTrainingData`.

Die lokalen Tabellen sind damit zugleich die **Datenquelle für den Dashboard-Tab** – keine doppelte Health-Connect-Abfrage für UI und Sync getrennt.

DB-Schema: `InitialLoadStateTable`

Kein Schedule-Eintrag, sondern reines Status-/Fortschritts-Tracking für den einmaligen 365-Tage-Backfill:

```
InitialLoadStateTable
├── id                -- lokaler PK
├── userId            FK -> UserDataTable.id, ON DELETE CASCADE
├── type              'HealthDataExport' | 'ExerciseDataExport'
├── status            'not_started' | 'in_progress' | 'completed' | 'failed'
├── startedAt
├── completedAt
└── lastSuccessfulChunkEndDate -- Resume-Punkt bei Abbruch (siehe unten)
```

Initial Load: Chunking beim Versand

365 Tage in einem HTTP-Request zu senden, kann zu großen Payloads führen. Lesen bleibt ein `readRecords`-Call mit Pagination (Health-Connect-seitig), der **Versand** an die API erfolgt in Chunks (z. B. monatsweise). Bei einem Fehler mitten im Initial Load wird über `lastSuccessfulChunkEndDate` beim nächsten Versuch ab dem letzten erfolgreichen Chunk fortgesetzt, statt bei Tag 1 neu zu beginnen.

HTTP

- Axios-Instanz mit Interceptor-Logik wie im Auth-Abschnitt beschrieben
 - Gleiche Instanz/Logik wird von UI **und** Background-Scheduler-Task genutzt (kein duplizierter Auth-Code)
-

Localization

- `i18next` + `react-i18next`
- Sprachen: Deutsch, Englisch

Struktur:

```
i18n.ts                -- Setup/Init von i18next
Resources/
├── en/
│   └── common.en.json
└── de/
    └── common.de.json
```

Custom Hook (kapselt `i18next`, keine direkten `useTranslation`-Aufrufe verstreut in Screens):

```
const { getResource, toggleLanguage } = useLocalization();

getResource('common.labelTest'): string
toggleLanguage(lang: 'en' | 'de'): void // persistiert Auswahl in SettingsTable
```

`toggleLanguage` schreibt die Auswahl in `SettingsTable.lang` (siehe Storage → DB-Schema); beim App-Start wird die gespeicherte Sprache aus `SettingsTable` gelesen und `i18next` entsprechend initialisiert.

Screens

- `LoginPage`
- `TabNavigation`: `Dashboard`, `Scheduler`

Scheduler-Tab-Inhalt: Pro Schedule (`HealthDataExport`, `ExerciseDataExport`) eine Zeile mit: `isActive`-Toggle, `intervalType` / hour / minute / day -Konfiguration, `lastRunAt` und `lastSuccessAt` als Klartext-Zeitstempel ("Zuletzt erfolgreich: ..."), sowie der Fehlerstatus aus dem Permission-Revocation-Handling (siehe `HealthConnectService`), falls vorhanden. Zusätzlich ein Bereich für den Initial Load: Zeitraum-Auswahl, Start-Button, Fortschrittsanzeige basierend auf `InitialLoadStateTable.status`.

Logout: Im `Dashboard`-Tab platziert. Löscht den Eintrag in `UserAuthenticationTable` für den aktuellen User (Tokens ungültig), navigiert zurück zu `LoginPage`. Andere User-Daten (`DailyHealthDataTable`, `ScheduleTable`, ...) bleiben erhalten – bei erneutem Login desselben Users sind sie weiter nutzbar. Kein vollständiges Löschen aller Userdaten beim Logout, das wäre ein separater "Account entfernen"-Vorgang.

Empfohlene Aufbau-Reihenfolge

1. Auth Flow strukturell in neue Architektur überführen (Referenz-Logik unverändert), AES-Key-Generierung/-Ablage im Keychain + Encrypt/Decrypt-Helper für `jwt` / `refreshToken` als Erstes einrichten
2. Axios-Instanz + Interceptor (inkl. Refresh-Mutex), liest/schreibt Tokens über `UserAuthenticationTable`
3. `HealthConnectService` (Verfügbarkeits-Check, Permission-Request, `readRecords` + Pagination), unabhängig testbar
4. Lokale DB anlegen: `UserDataTable`, `UserAuthenticationTable`, `SettingsTable`, `ScheduleTable`, `DailyHealthDataTable`, `ExerciseTable`, `ExerciseValuesTable`, `InitialLoadStateTable`
5. Payload-Typen/Struktur definieren (gemeinsam für Scheduler & Initial Load)
6. Scheduler-Mechanismus: Prototypen für Option A und B bauen, anhand der Evaluationskriterien entscheiden, dann Background-Permission-Flow + Heartbeat-Job final implementieren

7. Scheduled Sync (1-Tage-Snapshot) → `PostHealthData` + `PostTrainingData`
 8. Initial Load (365-Tage-Snapshot, gekunkter Versand, Resume-fähig) → dieselben Endpunkte
 9. Tab-Navigation (`Dashboard` , `Scheduler`) + Scheduler-Tab-UI (Schedule-Konfiguration, Status, Initial-Load-Trigger)
 10. Dashboard-Tab-Inhalt finalisieren
-

Explizit außerhalb dieses Scopes

- App-weite Performance-Optimierung über die hier genannten Punkte hinaus
 - Backend-seitige Aggregations-/Upsert-Implementierung (separates Ticket)
-

Bewusst vertagte Entscheidungen

- **Scheduler-Mechanismus** (Option A vs. B): wird per Prototyp-Vergleich entschieden, siehe Kapitel „Scheduler“
→ Implementierungsoptionen
- **`PostTrainingData` -Feldmodell**: wird zu einem späteren Zeitpunkt final definiert, siehe Kapitel „Endpunkte“